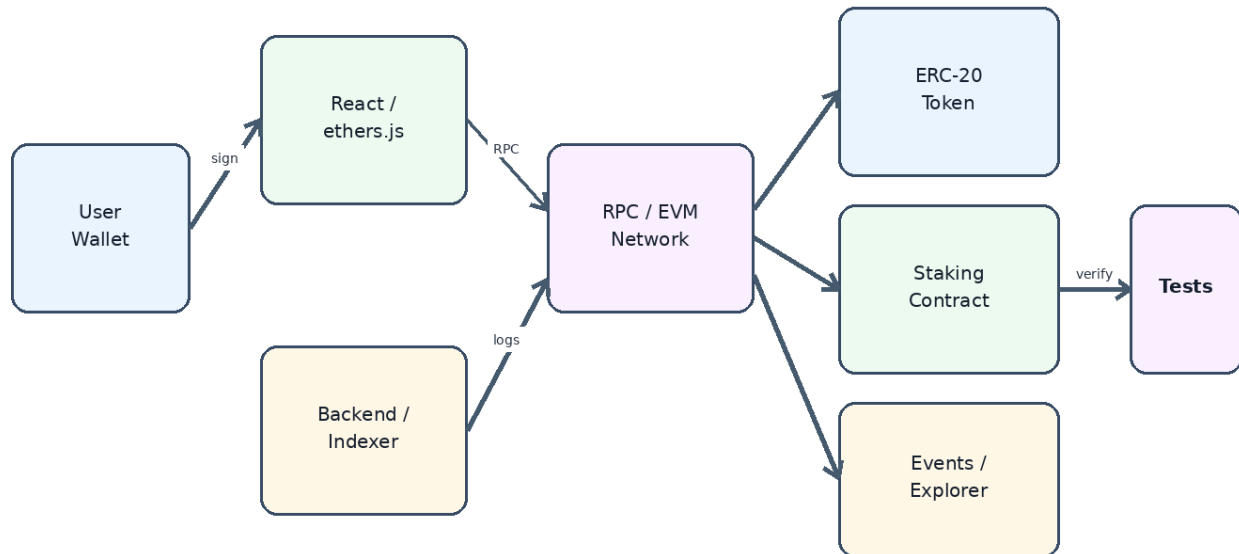


DAY 20

Blockchain Capstone + Interview Preparation

From Beginner Course to Portfolio-Ready Full-Stack Web3 Delivery

Day 20 Capstone - Full-Stack Staking dApp



60-minute objective: Integrate the complete 20-day skill stack into one defensible portfolio project and learn how to explain architecture, security, testing, deployment and tradeoffs in a client interview.

Capstone: an EVM staking dApp with Solidity contracts, Foundry tests, ethers.js frontend integration, optional event indexer, testnet deployment evidence, security notes and a professional handoff package.

1. Learning Objectives & Minute-by-Minute Plan

By the end of the final lecture, you should be able to describe and demonstrate an end-to-end blockchain project rather than answering with isolated definitions.

Time	Topic	Deliverable
0-5	Capstone brief + success criteria	Define exactly what “done” means.
5-12	Architecture + trust boundaries	Draw components and authoritative state.
12-23	Smart contract implementation	Review production-style staking contract.
23-32	Testing + security	Unit/fuzz/invariant and threat model.
32-39	Frontend + backend integration	Wallet/read/write/events/indexer flow.
39-45	Deploy + verify + handoff	Create evidence package.
45-54	Mock technical interview	Practice architecture/security answers.
54-58	Portfolio + proposal packaging	Turn capstone into proof.
58-60	30/60/90-day roadmap	Define next steps after the course.

Graduation standard: You do not need to know every protocol. You should be able to decompose a requirement, choose safe primitives, test assumptions, explain risk, and deliver verifiable evidence.

2. Capstone Client Brief

Simulated client request: “Build a staking dApp for our ERC-20 token. Users should connect a wallet, approve and stake tokens, see earned rewards, claim rewards and withdraw. Admins should configure funded reward campaigns and pause risky actions. Deliver a testnet deployment, tests and documentation.”

2.1 Questions before coding

- Which EVM chain and exact token addresses?
- Same token for stake/reward or separate tokens?
- Who funds rewards and how is insolvency prevented?
- Reward duration, rate rules and whether campaigns can overlap?
- Does pause block stake only, or also claim/withdraw?
- Which roles are client-controlled after deployment?
- Immutable contract or upgradeable requirement?
- Frontend framework, supported wallets and target browsers?
- Is a backend/indexer required, or can the UI read directly?
- What acceptance criteria and security review level are required?

2.2 Explicit non-goals

- No guaranteed APY/profit promises.
- No custody of client private keys or seed phrases.
- No unaudited mainnet release claim.
- No bridge, DAO or cross-chain features unless separately scoped.

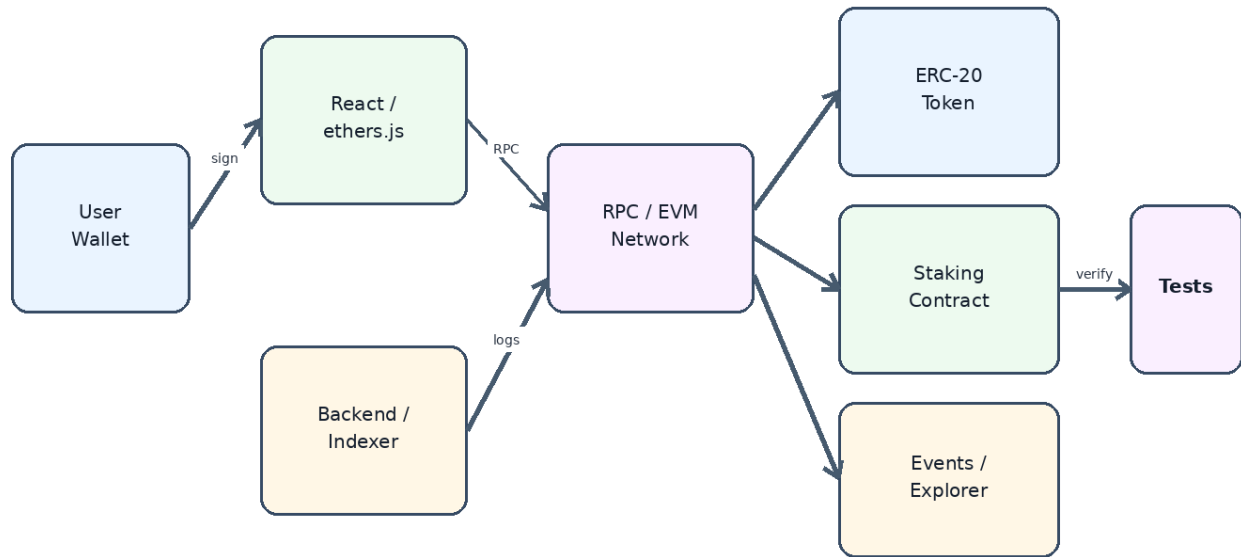
3. Requirements & Acceptance Criteria

ID	Requirement	Acceptance evidence
R1	User can stake >0 token amount	Foundry test + testnet tx
R2	User cannot withdraw more than recorded stake	Negative test
R3	Reward checkpoint updates before balance change	Reward time tests
R4	Only reward role can start campaign	Access-control test
R5	Campaign cannot promise more than funded rewards	Funding validation test
R6	Pause policy matches specification	Pause tests
R7	Frontend blocks unsupported chain	UI/network test
R8	Transactions show submitted/pending/confirmed/reverted states	Demo recording/screenshots
R9	Deployment is source-verified	Explorer verification
R10	Client receives runbook and role handoff evidence	HANDOFF.md + tx hashes

Traceability: Every important client requirement should map to code, tests and delivery evidence. This prevents “it seems to work” from becoming the acceptance standard.

4. System Architecture & Source of Truth

Day 20 Capstone - Full-Stack Staking dApp



Component	Responsibility	Trust note
Staking contract	Principal + reward accounting	Authoritative on-chain state
ERC-20 token	Stake/reward balances	Token behavior must be supported
Frontend	Wallet UX and contract calls	Never authoritative for balances
Wallet	User signature	Private keys remain user-controlled
RPC	Reads/broadcast/log access	Provider may fail or lag
Indexer	History/analytics	Rebuildable from chain logs where practical
Database	Off-chain cache	Must not override contract accounting

5. Contract State, Roles, Events & Errors

Roles:

```
DEFAULT_ADMIN_ROLE -> client governance / multisig  
REWARD_MANAGER_ROLE -> start funded reward campaigns  
PAUSER_ROLE -> pause selected risky paths
```

State:

```
_totalStaked  
balances[user]  
rewardRate  
rewardPerTokenStored  
lastUpdateTime  
periodFinish  
userRewardPerTokenPaid[user]  
rewards[user]
```

Events:

```
Staked, Withdrawn, RewardPaid, RewardAdded
```

Custom errors:

```
ZeroAmount, InsufficientStake, RewardTooHigh
```

Design rule: Use roles around actual responsibilities. Do not make a single owner silently control every production-sensitive action unless the client explicitly accepts that risk.

6. Capstone Solidity Contract - Part 1

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";

contract PortfolioStaking is AccessControl, ReentrancyGuard, Pausable {
    using SafeERC20 for IERC20;

    bytes32 public constant REWARD_MANAGER_ROLE = keccak256("REWARD_MANAGER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

    IERC20 public immutable stakingToken;
    IERC20 public immutable rewardsToken;

    uint256 public totalStaked;
    mapping(address => uint256) public balanceOf;

    uint256 public rewardRate;
    uint256 public rewardPerTokenStored;
    uint256 public lastUpdateTime;
    uint256 public periodFinish;
    mapping(address => uint256) public userRewardPerTokenPaid;
    mapping(address => uint256) public rewards;

    error ZeroAmount();
    error InsufficientStake();
    error RewardTooHigh();

    event Staked(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    event RewardPaid(address indexed user, uint256 reward);
    event RewardAdded(uint256 reward, uint256 duration);
```

6. Capstone Solidity Contract - Part 2

```

constructor(IERC20 stake_, IERC20 reward_, address admin_) {
    stakingToken = stake_;
    rewardsToken = reward_;
    _grantRole(DEFAULT_ADMIN_ROLE, admin_);
    _grantRole(REWARD_MANAGER_ROLE, admin_);
    _grantRole(PAUSER_ROLE, admin_);
}

modifier updateReward(address account) {
    rewardPerTokenStored = rewardPerToken();
    lastUpdateTime = lastTimeRewardApplicable();
    if (account != address(0)) {
        rewards[account] = earned(account);
        userRewardPerTokenPaid[account] = rewardPerTokenStored;
    }
    _;
}

function lastTimeRewardApplicable() public view returns (uint256) {
    return block.timestamp < periodFinish ? block.timestamp : periodFinish;
}

function rewardPerToken() public view returns (uint256) {
    if (totalStaked == 0) return rewardPerTokenStored;
    return rewardPerTokenStored
        + ((lastTimeRewardApplicable() - lastUpdateTime) * rewardRate * 1e18)
        / totalStaked;
}

function earned(address account) public view returns (uint256) {
    return (balanceOf[account] *
        (rewardPerToken() - userRewardPerTokenPaid[account])) / 1e18
        + rewards[account];
}

```


6. Capstone Solidity Contract - Part 3

```

function stake(uint256 amount)
    external
    nonReentrant
    whenNotPaused
    updateReward(msg.sender)
{
    if (amount == 0) revert ZeroAmount();
    totalStaked += amount;
    balanceOf[msg.sender] += amount;
    stakingToken.safeTransferFrom(msg.sender, address(this), amount);
    emit Staked(msg.sender, amount);
}

function withdraw(uint256 amount)
    public
    nonReentrant
    updateReward(msg.sender)
{
    if (amount == 0) revert ZeroAmount();
    if (amount > balanceOf[msg.sender]) revert InsufficientStake();
    totalStaked -= amount;
    balanceOf[msg.sender] -= amount;
    stakingToken.safeTransfer(msg.sender, amount);
    emit Withdrawn(msg.sender, amount);
}

function getReward() public nonReentrant updateReward(msg.sender) {
    uint256 reward = rewards[msg.sender];
    if (reward != 0) {
        rewards[msg.sender] = 0;
        rewardsToken.safeTransfer(msg.sender, reward);
        emit RewardPaid(msg.sender, reward);
    }
}

function exit() external {
    withdraw(balanceOf[msg.sender]);
    getReward();
}

```

6. Capstone Solidity Contract - Part 4

```
function notifyRewardAmount(uint256 reward, uint256 duration)
    external
    onlyRole(REWARD_MANAGER_ROLE)
    updateReward(address(0))
{
    if (duration == 0) revert ZeroAmount();

    if (block.timestamp >= periodFinish) {
        rewardRate = reward / duration;
    } else {
        uint256 remaining = periodFinish - block.timestamp;
        uint256 leftover = remaining * rewardRate;
        rewardRate = (reward + leftover) / duration;
    }

    uint256 available = rewardsToken.balanceOf(address(this));
    if (address(stakingToken) == address(rewardsToken)) {
        if (available < totalStaked) revert RewardTooHigh();
        available -= totalStaked;
    }
    if (rewardRate > available / duration) revert RewardTooHigh();

    lastUpdateTime = block.timestamp;
    periodFinish = block.timestamp + duration;
    emit RewardAdded(reward, duration);
}

function pause() external onlyRole(PAUSER_ROLE) { _pause(); }
function unpause() external onlyRole(PAUSER_ROLE) { _unpause(); }
}
```

Teaching limitation: This capstone is designed to demonstrate architecture and testing. Production deployments still require token-behavior assumptions, governance policy, independent review where appropriate, and careful mainnet release procedures.

7. Reward Accounting - Explain It in an Interview

The scalable pattern stores a global cumulative reward-per-token checkpoint. Each user stores the checkpoint value already accounted for. This avoids iterating over every staker when time passes.

```
global rewardPerToken +=
    elapsedSeconds * rewardRate * 1e18 / totalStaked

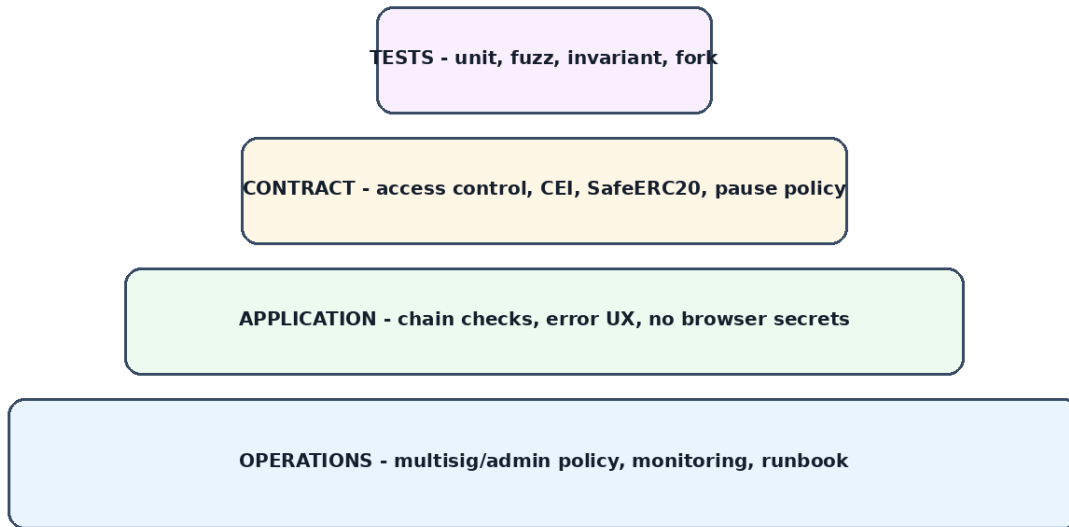
user.earned +=
    userBalance * (globalRPT - userPaidRPT) / 1e18
```

1. Update the global checkpoint using the old totalStaked.
2. Set the user's accrued reward using the old user balance.
3. Record the user checkpoint.
4. Only then change stake principal.

Interview signal: Do not merely recite the formula. Explain why reward state must be checkpointed before modifying principal and why looping over users is not scalable.

8. Capstone Threat Model & Security Review

Capstone Security Layers

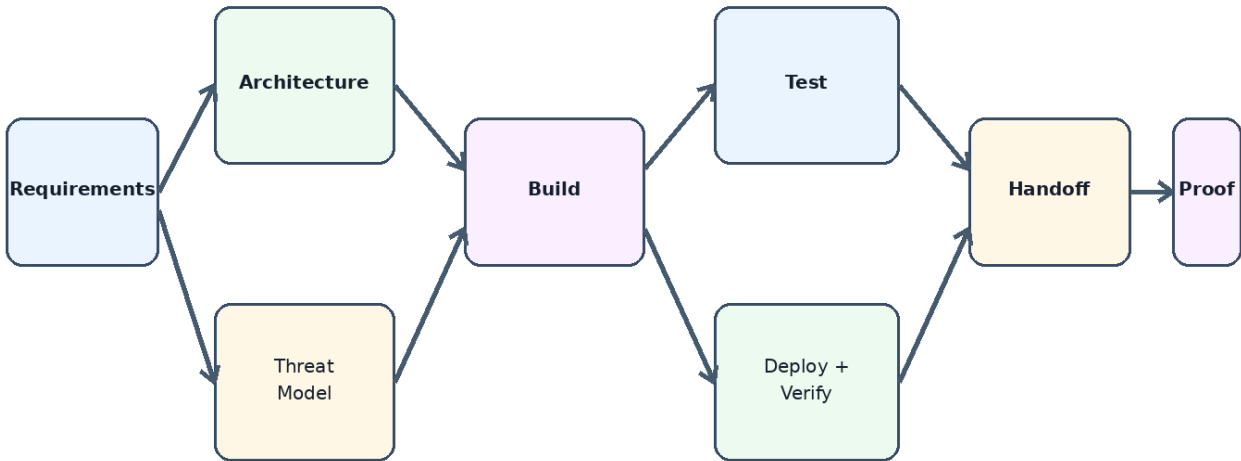


Risk	Mitigation / decision
Reentrancy during transfers	CEI-style accounting + ReentrancyGuard
Non-standard ERC-20 return values	SafeERC20
Unauthorized reward campaigns	AccessControl roles
Reward insolvency	Validate funded amount and reserved principal
Admin key compromise	Client multisig / role separation
Paused funds trapped	Withdraw remains available; document policy
Fee-on-transfer/rebasing token	Reject or explicitly design support
Wrong-chain frontend	Chain ID + address registry checks
RPC outage/reorg	Retries, confirmations, cursor reconciliation
Unverified deployment	Verify source and preserve release metadata

9. Test Strategy: Unit -> Fuzz -> Invariant -> Acceptance

Layer	Examples
Unit	stake/withdraw/claim, events, roles, pause
Negative	zero amount, over-withdraw, unauthorized reward, underfunded campaign
Time-based	reward accrual before/during/after campaign
Fuzz	random bounded stake/withdraw/reward values
Invariant	sum/model of stake equals accounting; no user withdraws beyond credit
Fork	optional integration with real external token/protocol state
Acceptance	wallet flow + verified testnet deployment

Capstone Delivery Pipeline



10. Foundry Tests - Core Examples

```
function testStakeAndWithdraw() public {
    vm.startPrank(alice);
    staking.stake(100 ether);
    assertEq(staking.balanceOf(alice), 100 ether);
    staking.withdraw(40 ether);
    assertEq(staking.balanceOf(alice), 60 ether);
    vm.stopPrank();
}

function testUnauthorizedRewardManagerReverts() public {
    vm.prank(alice);
    vm.expectRevert();
    staking.notifyRewardAmount(700 ether, 7 days);
}

function testRewardAccrues() public {
    rewardToken.mint(address(staking), 700 ether);
    staking.notifyRewardAmount(700 ether, 7 days);
    vm.prank(alice); staking.stake(100 ether);
    vm.warp(block.timestamp + 1 days);
    assertGt(staking.earned(alice), 0);
}
```

11. Fuzz & Invariant Thinking

```
function testFuzzStake(uint96 raw) public {
    uint256 amount = bound(uint256(raw), 1, 1_000 ether);
    vm.prank(alice);
    staking.stake(amount);
    assertEq(staking.balanceOf(alice), amount);
}

// Useful properties:
// - totalStaked never decreases except through valid withdrawal
// - user principal cannot be created by claiming rewards
// - unauthorized callers cannot alter reward schedule
// - tracked principal remains solvent for supported standard tokens
```

11.1 What invariants are not

An invariant is not “the test passed once.” It is a property that should remain true across many generated call sequences and state transitions.

12. Frontend Integration with ethers.js v6

```
import { BrowserProvider, Contract, parseUnits, formatUnits } from "ethers";

const provider = new BrowserProvider(window.ethereum);
await provider.send("eth_requestAccounts", []);
const signer = await provider.getSigner();
const network = await provider.getNetwork();

if (network.chainId !== EXPECTED_CHAIN_ID) throw new Error("Wrong network");

const token = new Contract(TOKEN, ERC20_ABI, signer);
const staking = new Contract(STAKING, STAKING_ABI, signer);
const amount = parseUnits("100", 18);

const approveTx = await token.approve(STAKING, amount);
await approveTx.wait();

const stakeTx = await staking.stake(amount);
const receipt = await stakeTx.wait();
console.log(receipt.hash);

const earned = await staking.earned(await signer.getAddress());
console.log(formatUnits(earned, 18));
```

Provider vs Signer: A Provider is the read-oriented blockchain connection. A Signer authorizes state-changing actions for an account. In ethers v6, BrowserProvider wraps injected EIP-1193 providers such as wallet extensions.

13. Transaction UX Is Part of Correctness

State	UI behavior
Disconnected	Show connect-wallet action
Unsupported chain	Explain expected chain and safe switch
Approval needed	Show approval as a separate transaction
Wallet prompt	Do not claim transaction submitted yet
Submitted	Show hash immediately
Pending	Disable duplicate action and show pending state
Confirmed	Refresh authoritative reads
Rejected	User-friendly rejection message
Reverted	Decode known custom errors where possible
RPC failure	Retry reads; do not assume transaction failed if hash exists

A blockchain frontend must distinguish wallet rejection, transaction submission, mining/confirmation and contract reversion. These are different states with different recovery actions.

14. Optional Backend / Event Indexer

```
const stakingRead = new Contract(STAKING, STAKING_ABI, provider);

stakingRead.on("Staked", async (user, amount, event) => {
  const log = event.log;
  await store.upsert({
    key: `${log.transactionHash}:${log.index}`,
    blockNumber: log.blockNumber,
    user,
    amount: amount.toString(),
    type: "STAKED"
  });
});
```

Reliability concern	Design response
WebSocket missed event	Historical backfill from persistent block cursor
Duplicate delivery	Idempotent event key
Reorg	Confirmation policy + reconciliation
Provider throttling	Backoff + bounded block ranges
DB loss	Rebuild from chain where practical
Private signer	Keep server-side; prefer client-controlled governance for admin actions

15. Professional Repository & Documentation Package

```
portfolio-staking/  
  contracts/  
    src/PortfolioStaking.sol  
    test/PortfolioStaking.t.sol  
    script/Deploy.s.sol  
    foundry.toml  
  frontend/  
    src/  
    package.json  
    .env.example  
  backend/          # optional  
    src/indexer.ts  
  deployments/  
    sepolia.json  
  docs/  
    REQUIREMENTS.md  
    ARCHITECTURE.md  
    SECURITY.md  
    TEST_PLAN.md  
    HANDOFF.md  
  README.md  
  .github/workflows/ci.yml
```

Portfolio quality: A reviewer should be able to understand the goal, run the tests, find the deployment, inspect security assumptions and reproduce the basic flow without interviewing you first.

16. Deployment, Verification & Release Record

1. Freeze a release commit/tag.
2. Run `forge fmt --check`, `forge build`, `forge test` and relevant fuzz/invariant suites.
3. Review constructor args, token addresses, chain ID and role recipients.
4. Deploy with a scripted process.
5. Verify source on the target explorer.
6. Run smoke tests: read config, stake small amount, claim/withdraw, unauthorized admin call.
7. Record deployment transaction, block, contract address and release commit.
8. Transfer/confirm production roles according to the client runbook.

```
{  
  "chainId": 11155111,  
  "release": "v1.0.0",  
  "commit": "abc1234",  
  "contract": "PortfolioStaking",  
  "address": "0x...",  
  "deploymentTx": "0x...",  
  "stakingToken": "0x...",  
  "rewardsToken": "0x...",  
  "admin": "0x...",  
  "verified": true  
}
```

17. Client Handoff Checklist

Artifact	What the client receives
Source release	Tagged/committed source
Tests	Commands + result summary
Deployment record	Chain, addresses, tx hashes, inputs
Verification	Explorer source verification
Role map	Who can reward/pause/admin
Runbook	How to fund campaign, pause/unpause, diagnose common failures
Frontend config	.env.example and deployment registry
Security notes	Threats, assumptions, unsupported token behaviors
Known limitations	Explicit future work / non-goals
Support boundary	What is included after handoff

18. Turn the Capstone into a Portfolio Case Study

TITLE

Full-Stack EVM Staking dApp - Solidity + Foundry + ethers.js

PROBLEM

Users need to stake an ERC-20 and earn a funded reward campaign.

SOLUTION

Role-controlled staking contract with reward checkpoints, tests, wallet UI and verified testnet deployment.

SECURITY

SafeERC20, ReentrancyGuard, AccessControl, funded reward validation, wrong-chain frontend checks and documented admin policy.

EVIDENCE

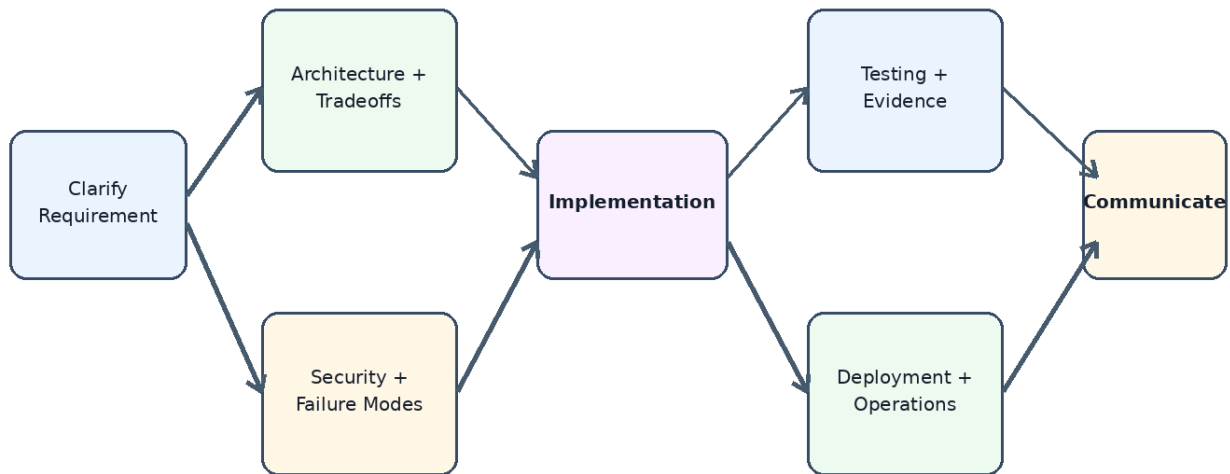
- GitHub repository
- forge test output
- verified testnet address
- deployment JSON
- screenshots / short demo
- architecture + security docs

LIMITATIONS

Teaching/portfolio project; production launch requires project-specific review.

19. Mock Technical Interview - How to Structure Answers

Blockchain Interview Answer Loop



A strong answer usually follows: clarify -> propose architecture -> identify risks/tradeoffs -> implementation -> testing -> deployment/operations -> communicate assumptions.

19.1 Example question

"How would you build a token staking system?"

1. Clarify reward economics, token behavior, lock rules, chain, roles and funding.
2. Choose contract state and a scalable reward checkpoint model.
3. Use standard security primitives and define pause/admin policy.
4. Test happy paths, negative paths, time, fuzz and invariants.
5. Integrate frontend with chain/address validation and transaction states.
6. Deploy reproducibly, verify, smoke-test and hand off roles/documentation.

20. 20 Core Blockchain Interview Questions

#	Question	What a strong answer should mention
1	EOA vs contract account?	Key-controlled vs code-controlled account/state.
2	What is gas?	Execution resource accounting and transaction fee mechanics.
3	memory vs storage vs calldata?	Lifetime, mutability, cost/use context.
4	ERC-20 allowance?	approve + transferFrom delegated spending.
5	Reentrancy?	External call before state finalization; CEI/guard.
6	Why SafeERC20?	Handle token call return behavior safely.
7	Event vs storage?	Logs for off-chain observation; storage for contract state.
8	What is an ABI?	Encoding/interface for calls and events.
9	Provider vs Signer?	Read connection vs authorized writes.
10	Why fuzz tests?	Explore many input values automatically.
11	What is an invariant?	Property that should remain true across state transitions.
12	What is a proxy risk?	delegatecall/storage layout/admin upgrade authority.
13	Oracle freshness?	Validate updatedAt/heartbeat assumptions.
14	AMM $x*y=k$?	Reserve relationship and price impact basics.
15	Slippage vs price impact?	Pool movement vs user execution tolerance.
16	Flash loan atomicity?	Borrow/use/repay in one transaction or revert.
17	PDA on Solana?	Deterministic off-curve address controlled by program seeds.
18	Why chain ID checks?	Prevent calls to wrong network/address set.
19	How debug revert?	Receipt/trace/revert data, caller, state, reproduce test.
20	How deliver safely?	Tests, verification, deployment record, role handoff, runbook.

21. Client Scenario Drills

Client says...	Your response process
"Stake transaction fails."	Confirm chain/address, balance, allowance, pause state, amount; inspect revert; reproduce test.
"Rewards are wrong."	Write expected formula, timestamp window, reward funding and user checkpoints; reproduce deterministic time test.
"Can you deploy to BSC too?"	Check EVM compatibility, dependencies, token addresses, chain config, gas asset, explorer verification; redeploy and test.
"Make it upgradeable."	Clarify reason, governance, proxy pattern, storage layout, initializer and upgrade tests before changing architecture.
"Can frontend keep admin private key?"	No. Browser secrets are extractable; use client-controlled wallet/multisig or secure backend signer if explicitly designed.
"Guarantee this arbitrage will profit."	Do not guarantee; model fees/slippage/gas/MEV and execute only when validated conditions are met.

22. Day 20 Practical Exam - 25-Point Rubric

Area	Points	Pass evidence
Requirements	3	10+ explicit requirements + acceptance criteria
Architecture	3	Diagram + source-of-truth/trust-boundary explanation
Solidity	5	Working staking/reward contract with roles/events/errors
Testing	5	Unit + negative + fuzz + at least one invariant idea
Frontend	3	Wallet, chain check, approve/stake/read/claim/withdraw flow
Deployment	2	Testnet address + verification + deployment metadata
Security	2	Threat model + privileged-role/solvency notes
Handoff	2	README + SECURITY + HANDOFF + known limitations

Suggested pass mark: 18/25 with no critical security misunderstanding. A lower score means revisit the relevant course days before pitching that capability to clients.

23. Final Knowledge Check - 15 Questions

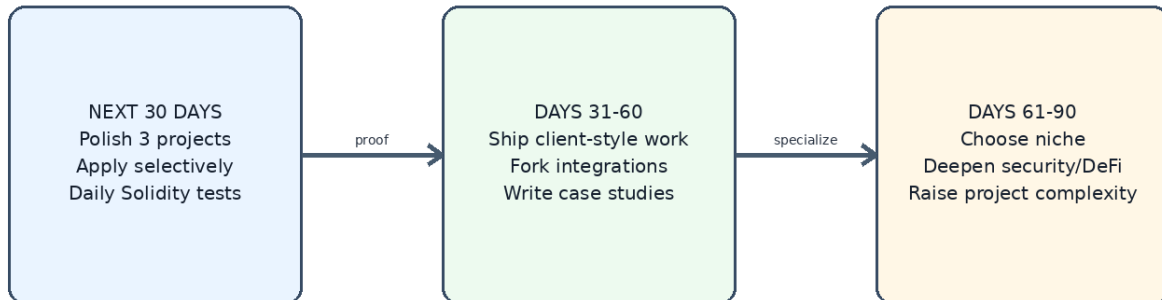
1. Why is the smart contract, not the database, the source of truth for stake balances?
2. Why checkpoint rewards before changing a user balance?
3. What problem does ReentrancyGuard reduce?
4. Why use SafeERC20?
5. What must happen before transferFrom can usually move a user ERC-20?
6. What is the purpose of a chain ID check in the frontend?
7. What is the difference between submitted and confirmed transaction states?
8. Why store a persistent block cursor in an indexer?
9. What is a reorg and why can it affect event processors?
10. What is an invariant test?
11. What deployment evidence should be recorded?
12. Why should production admin roles be explicitly handed off?
13. What is the difference between an audit and your own developer security review?
14. How should you respond when a client changes scope after work begins?
15. What three pieces of proof make this capstone credible to a client?

23.1 Short answer key

1. On-chain code/state determines canonical custody/accounting; DB is an off-chain view.
2. So accrual uses the old balance for elapsed time.
3. Reentrant nested calls around sensitive state/external interactions.
4. Safer ERC-20 interactions across token return conventions.
5. Sufficient allowance/approval plus balance.
6. Avoid wrong-network calls and incorrect addresses.
7. Submitted means broadcast/hash exists; confirmed means included/mined to chosen confirmation depth.
8. Recover missed events and resume deterministically.
9. Canonical chain can replace recent blocks/logs; processor must reconcile.
10. A property expected to remain true across generated sequences.
11. Release commit, chain ID, address, tx hash, inputs, compiler/settings, verification.
12. Client must know and control privileged authority according to agreed governance.
13. Developer review is not an independent formal assurance engagement.
14. Document impact and agree change/milestone before implementation.
15. Tests, verified deployment, readable repo/docs/demo.

24. After the Course: 30 / 60 / 90 Day Roadmap

After Day 20 - 30 / 60 / 90 Day Roadmap



Period	Technical goal	Freelance goal
Next 30 days	Polish staking, token/NFT and transaction-debug projects; write tests daily.	Publish 3 strong case studies; send selective proposals to tightly matched jobs.
Days 31-60	Fork-test a real DeFi integration; deepen access control/upgrades/oracles.	Complete small client-style deliveries; improve proposal and estimation based on feedback.
Days 61-90	Choose specialization: DeFi integration, Solidity contracts, Web3 frontend/backend, or security-focused development.	Raise project complexity/rate only when evidence supports it; build referrals and repeatable process.

Expertise is cumulative: Twenty days creates a structured foundation. Senior/expert capability comes from repeated production-like projects, debugging unfamiliar systems, security discipline, protocol reading and successful client delivery over time.

25. Day 20 Capstone Cheat Sheet

CLIENT FLOW

Brief -> Questions -> Requirements -> Acceptance -> Architecture -> Threat model
-> Build -> Tests -> Deploy -> Verify -> Handoff -> Evidence

SMART CONTRACT

SafeERC20 | AccessControl | ReentrancyGuard | Pausable
reward checkpoint BEFORE principal change
fund rewards; protect principal

TESTING

unit | revert | event | time | fuzz | invariant | fork | acceptance

FRONTEND

BrowserProvider -> Signer -> Contract
chain ID -> allowance -> approve -> stake -> wait -> refresh

BACKEND

logs -> confirmations -> dedupe -> cursor -> backfill -> reconcile

DELIVERY

release commit | chain ID | contract address | deployment tx | verification
roles | test report | runbook | known limitations

INTERVIEW

clarify -> architecture -> risks -> implementation -> tests -> deployment -> communicate

26. Instructor Speaking Guide - 60-Minute Delivery

Minute	Instructor action
0-5	Read client brief. Ask students to identify missing requirements before coding.
5-12	Draw architecture. Ask which component is authoritative for funds and why.
12-18	Walk through state/roles/events. Emphasize privilege and reward funding.
18-23	Explain reward checkpoint math and stake/withdraw ordering.
23-28	Review reentrancy, token behavior, solvency and pause policy.
28-32	Write/review core Foundry tests; ask for two negative and one invariant property.
32-36	Trace approve -> stake -> wait -> refresh in ethers.js.
36-39	Explain event indexer, cursor, duplicates and reorgs.
39-45	Walk deployment checklist, verification and handoff.
45-50	Ask five interview questions; require structured answers.
50-54	Run one debugging scenario and ask for evidence-first process.
54-58	Turn project into portfolio case study and proposal proof.
58-60	Assign practical exam and 30/60/90-day plan.

27. Primary Technical References - Verified August 2026

Use primary documentation when implementation details change. The course examples are intentionally based on standard, documented primitives rather than copying unaudited snippets.

- OpenZeppelin Contracts 5.x - <https://docs.openzeppelin.com/contracts/5.x>
- Foundry - <https://www.getfoundry.sh/>
- Foundry Testing - <https://www.getfoundry.sh/forging/testing>
- Foundry Scripting - <https://www.getfoundry.sh/forging/scripting>
- ethers.js v6 Getting Started - <https://docs.ethers.org/v6/getting-started/>
- ethers.js v6 Providers - <https://docs.ethers.org/v6/api/providers/>
- Ethereum Smart Contracts - <https://ethereum.org/en/developers/docs/smart-contracts/>
- Ethereum Smart Contract Testing - <https://ethereum.org/en/developers/docs/smart-contracts/testing/>
- Ethereum Smart Contract Security - <https://ethereum.org/en/developers/docs/smart-contracts/security/>
- Ethereum JSON-RPC - <https://ethereum.org/en/developers/docs/apis/json-rpc/>

Version awareness: Tooling, library APIs, testnets and platform policies change. Recheck current official documentation before production use, especially deployment addresses, chain configuration and security-sensitive library APIs.

28. Course Graduation Checklist

- I can explain a blockchain transaction from signature to confirmation.
- I can write, test and deploy Solidity contracts using standard libraries.
- I can implement ERC-20/NFT and staking-style patterns at portfolio level.
- I can use Foundry tests, fuzzing and invariants to challenge my code.
- I can identify common smart-contract security risks and privileged trust assumptions.
- I can connect a Web3 frontend using ethers.js and manage transaction UX states.
- I can build an event-monitoring backend with confirmations, cursors and deduplication.
- I understand AMMs, staking, lending, vault and flash-loan architecture at a developer level.
- I can consume oracle data with validation and explain automation architecture.
- I can distinguish EVM portability from a Solana architecture rewrite.
- I can turn a client brief into scope, milestones, acceptance evidence and handoff.
- I can present a credible portfolio without exaggerating expertise.

DAY 20 COMPLETE - BEGIN BUILDING REAL EVIDENCE

The next stage is repetition: build, test, deploy, debug, document, review, and gradually take on higher-risk work only when your evidence and judgment support it.